

React Native with TypeScript:

Advanced Tutorial

This combines the advanced RN topics (performance, animations, state at scale, native modules, testing, CI/CD) with TypeScript typing patterns throughout. Assumes you're comfortable with typed props, hooks, and typed navigation already.

1. Typed Performance Patterns

Generic memoized components

```
import { memo } from 'react';
import { Pressable, Text } from 'react-native';

interface ListItemProps<T> {
  item: T;
  getLabel: (item: T) => string;
  onPress: (item: T) => void;
}

function ListItemInner<T>({ item, getLabel,
onPress }: ListItemProps<T>) {
  return (
    <Pressable onPress={() => onPress(item)}>
      <Text>{getLabel(item)}</Text>
    </Pressable>
  );
}
```

```
// memo() erases generics, so cast the result
back to a generic function type
export const ListItem = memo(ListItemInner) as
typeof ListItemInner;
```

This pattern — `memo(Component) as typeof Component` — is the standard workaround for `memo`'s TypeScript limitation of stripping generic type parameters.

Typed `useCallback` / `useMemo`

```
interface Product {
  id: string;
  price: number;
}

function ProductList({ products }: { products:
Product[] }) {
  const handlePress = useCallback((id: string):
void => {
    console.log('pressed', id);
  }, []);

  const sorted: Product[] = useMemo(
    () => [...products].sort((a, b) => a.price -
b.price),
    [products]
  );

  return (
    <FlatList<Product>
      data={sorted}
      keyExtractor={({item}) => item.id}
      renderItem={({ item }) => <ListItem item=
{item} getLabel={(p) => `$$${p.price}`} onPress=
```

```

    {handlePress} />
  />
);
}

```

Typed `getItemLayout`

```

const ITEM_HEIGHT = 60;

const getItemLayout = (
  data: Product[] | null | undefined,
  index: number
): { length: number; offset: number; index:
number } => ({
  length: ITEM_HEIGHT,
  offset: ITEM_HEIGHT * index,
  index,
});

```

Typing this signature exactly as `FlatList` expects avoids a common source of silent `any` leaking into your list logic.

2. Typed `Reanimated`

Shared values need explicit generics when their type isn't obvious from the initial value:

```

import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
  useAnimatedGestureHandler,
} from 'react-native-reanimated';
import { PanGestureHandler,

```

```

PanGestureHandlerGestureEvent } from 'react-
native-gesture-handler';

interface GestureContext {
  startX: number;
  [key: string]: unknown; // required index
signature for Reanimated's context object
}

function DraggableCard() {
  const translateX = useSharedValue<number>(0);

  const animatedStyle = useAnimatedStyle(() =>
({
  transform: [{ translateX: translateX.value
}],
})),
  }));

  const gestureHandler =
useAnimatedGestureHandler<
  PanGestureHandlerGestureEvent,
  GestureContext
>({
  onStart: (_, ctx) => {
    ctx.startX = translateX.value;
  },
  onActive: (event, ctx) => {
    translateX.value = ctx.startX +
event.translationX;
  },
  onEnd: () => {
    translateX.value = withSpring(0);
  },
});

  return (
    <PanGestureHandler onGestureEvent=

```

```

{gestureHandler}>
    <Animated.View style={[styles.card,
animatedStyle]} />
    </PanGestureHandler>
  );
}

```

The three type parameters on

`useAnimatedGestureHandler<Event, Context>` catch a very common bug: referencing a context property (`ctx.startX`) that was never initialized in `onStart`, or mistyping the event's payload shape.

3. Typed State Management at Scale

Zustand with full type inference

```

import { create } from 'zustand';

interface CartItem {
  id: string;
  name: string;
  price: number;
}

interface CartState {
  items: CartItem[];
  addItem: (item: CartItem) => void;
  removeItem: (id: string) => void;
  total: () => number;
}

const useCartStore = create<CartState>((set,
get) => ({

```

```

    items: [],
    addItem: (item) => set((state) => ({ items:
[...state.items, item] })),
    removeItem: (id) =>
      set((state) => ({ items:
state.items.filter((i) => i.id !== id) })),
    total: () => get().items.reduce((sum, i) =>
sum + i.price, 0),
  }));

// Selector is fully typed: `state` is inferred
as CartState
function CartBadge() {
  const itemCount = useCartStore((state) =>
state.items.length);
  return <Text>{itemCount} items</Text>;
}

```

Redux Toolkit with typed hooks

Redux Toolkit + TypeScript requires typed versions of `useSelector` / `useDispatch` so every component gets full inference instead of `any`:

```

// store.ts
import { configureStore } from
 '@reduxjs/toolkit';
import cartReducer from './cartSlice';

export const store = configureStore({ reducer: {
  cart: cartReducer } });

export type RootState = ReturnType<typeof
store.getState>;
export type AppDispatch = typeof store.dispatch;

```

```
// hooks.ts
import { useDispatch, useSelector,
TypedUseSelectorHook } from 'react-redux';
import type { RootState, AppDispatch } from
'./store';

export const useAppDispatch = () =>
useDispatch<AppDispatch>();
export const useAppSelector:
TypedUseSelectorHook<RootState> = useSelector;
```

```
// cartSlice.ts
import { createSlice, PayloadAction } from
'@reduxjs/toolkit';

interface CartItem { id: string; name: string;
price: number; }
interface CartState { items: CartItem[]; }

const initialState: CartState = { items: [] };

const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: {
    addItem: (state, action:
PayloadAction<CartItem>) => {
      state.items.push(action.payload);
    },
    removeItem: (state, action:
PayloadAction<string>) => {
      state.items = state.items.filter((i) =>
i.id !== action.payload);
    },
  },
});
```

```
export const { addItem, removeItem } =  
  cartSlice.actions;  
export default cartSlice.reducer;
```

Now `useAppSelector((state) => state.cart.items)`
autocompletes fully, and `dispatch(addItem(wrongShape))`
fails to compile if `wrongShape` doesn't match `CartItem`.

4. Typed Native Modules (Expo Modules API)

The Expo Modules API generates a typed JS interface automatically from your native definition, but you still declare the TS contract explicitly for consumers:

```
// src/MyNativeModule.types.ts  
export interface MyNativeModuleEvents {  
  onValueChange: (event: { value: number }) =>  
    void;  
}  
  
export interface MyNativeModule {  
  getDeviceModel(): string;  
  getBatteryLevelAsync(): Promise<number>;  
  addListener<K extends keyof  
MyNativeModuleEvents>(  
    eventName: K,  
    listener: MyNativeModuleEvents[K]  
  ): { remove: () => void };  
}
```



```
// src/index.ts
import { requireNativeModule } from 'expo-
modules-core';
import type { MyNativeModule } from
'./MyNativeModule.types';

export default
requireNativeModule<MyNativeModule>
('MyNativeModule');
```

Consumers get full type checking and autocomplete on native calls, including async return types and event listener payloads — critical when native and JS teams work separately and shouldn't need to cross-reference native source to know a function's signature.

5. Typed Testing

Component tests with typed mocks

```
import { render, screen, fireEvent } from
'@testing-library/react-native';
import Counter from './Counter';

jest.mock('./useAnalytics', () => ({
  useAnalytics: (): { track: jest.Mock } => ({
    track: jest.fn() }),
}));

test('increments count when button is pressed',
() => {
  render(<Counter />);
  fireEvent.press(screen.getByText('Add One'));
```

```
expect(screen.getByText('1')).toBeTruthy();
});
```

Typing API mocks precisely

```
interface Post {
  id: number;
  title: string;
}

const mockPosts: Post[] = [{ id: 1, title: 'Test
Post' }];

global.fetch = jest.fn(() =>
  Promise.resolve({
    ok: true,
    json: () => Promise.resolve(mockPosts),
  })
) as jest.Mock;
```

Typing the mock's return shape against the real `Post` interface means a schema change in production code immediately surfaces as a test-mock mismatch, rather than a runtime surprise.

6. Typed Navigation at Scale (Nested Navigators)

Real apps combine stack, tab, and drawer navigators — typing this correctly avoids a common source of `any` creep as the app grows.

```

import { NavigatorScreenParams } from '@react-
navigation/native';

// Each navigator gets its own param list
type HomeStackParamList = {
  Feed: undefined;
  PostDetail: { postId: string };
};

type ProfileStackParamList = {
  Profile: undefined;
  Settings: undefined;
};

// The tab navigator nests entire stacks as
screens
type RootTabParamList = {
  HomeTab:
NavigatorScreenParams<HomeStackParamList>;
  ProfileTab:
NavigatorScreenParams<ProfileStackParamList>;
};

// Composite type for a screen that needs to
navigate OUT of its own stack
import { CompositeScreenProps } from '@react-
navigation/native';
import { NativeStackScreenProps } from '@react-
navigation/native-stack';
import { BottomTabScreenProps } from '@react-
navigation/bottom-tabs';

type PostDetailProps = CompositeScreenProps<
  NativeStackScreenProps<HomeStackParamList,
  'PostDetail'>,
  BottomTabScreenProps<RootTabParamList>
>;

```

```
function PostDetailScreen({ navigation, route }:
PostDetailProps) {
  const { postId } = route.params; // typed to
HomeStackParamList
  // navigation can now also jump to ProfileTab,
typed to RootTabParamList
  return <Text>Post {postId}</Text>;
}
```

`CompositeScreenProps` is the piece most teams miss — without it, TypeScript only knows about the immediate stack's routes, and cross-navigator navigation calls silently fall back to `any`.

7. Runtime Validation at the TypeScript Boundary

TypeScript types vanish at runtime — an API can still send malformed data that satisfies the compiler but crashes the app. **Zod** closes that gap:

```
import { z } from 'zod';

const ProductSchema = z.object({
  id: z.number(),
  title: z.string(),
  price: z.number().positive(),
});

type Product = z.infer<typeof ProductSchema>; //
type derived from the schema, not duplicated

async function fetchProducts():
```

```

Promise<Product[]> {
  const response = await
  fetch('https://fakestoreapi.com/products');
  const data: unknown = await response.json();
  return z.array(ProductSchema).parse(data); //
  throws if shape doesn't match
}

```

Deriving the `Product` type from the schema (`z.infer`) means your validation and your type definition can never drift apart — a common bug when they're maintained separately.

8. CI: Type Checking as a Gate

```

# .github/workflows/ci.yml
name: CI
on: [pull_request]
jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx tsc --noEmit          # type
        check without building
      - run: npx eslint . --max-warnings 0
      - run: npx jest --coverage

```

`tsc --noEmit` as its own CI step (separate from bundling) catches type errors fast without waiting on a full EAS build — put it first so a broken PR fails in seconds, not minutes.

Common Pitfalls at This Level

- **Don't type Redux/Zustand selectors as `any` "just to unblock a PR."** It defeats the entire purpose and tends to spread — one untyped selector often becomes ten.
- **`as` casts in native module bridges are a code smell.** If you're casting a native return value to force it to match your interface, your `.types.ts` file is wrong — fix the contract, not the call site.
- **Reanimated worklets can't close over non-shared-value state safely** — TypeScript won't catch this at compile time, so structure state explicitly as shared values rather than relying on plain component state inside a worklet.

Next Steps

- Push into masters-level internals (JSI, Fabric, Metro) once these typed patterns feel automatic — the masters tutorial covers those without a TypeScript-specific lens, but every pattern there can be typed the same way.
- Look at **`ts-pattern`** for typed exhaustive matching over union types (useful for typed navigation state or API response variants).
- Adopt `strict: true` and `noUncheckedIndexedAccess` in `tsconfig.json` for the strongest guarantees as your team scales.